

Human developers must translate natural language requirements into structured workflows, which is time-consuming and error-prone. This raises a natural question: **can agentic workflows be automatically generated from natural language?** Achieving this is challenging for two reasons. First, real-world requirements are often complex and implicit, making it difficult to infer correct control flow and tool usage from natural language alone. Second, user requirements frequently change, requiring workflows to be revised or regenerated while preserving correctness and consistency.

To systematically study this, we introduce **Chat2Workflow** (MIT license), the first benchmark focused on generating executable visual workflows from natural language (Figure 1). It comprises 237 human-annotated instances across six domains (Figure 2) to evaluate whether workflows meet the expected requirements for execution. Each round is equipped with realistic user requests and target workflow components across diverse domains and complexity levels.

The benchmark evaluates a model’s ability to infer structures, select tools, and produce executable, intent-aligned workflows. Experiments show that current state-of-the-art language models can often capture high-level intent but struggle to generate correct and stable workflows, especially as tasks become more complex or requirements evolve. To address this, we propose an error-driven agentic baseline with auto-repair mechanisms. Although it improves absolute resolve rates by 6.05%, the remaining performance gap highlights executable workflow generation as a key automation bottleneck, positioning Chat2Workflow as a crucial benchmark for future research in structured reasoning and adaptive workflow synthesis.

2 Constructing Benchmark

2.1 Workflow Orchestration Platforms

Overview Dify and Coze are mainstream platforms for building agentic workflows by connecting functional nodes. These workflows are visualized as Directed Acyclic Graphs on the frontend and stored as structured YAML files on the backend. The platforms execute tasks node-by-node, automatically handling tool invocations, inference, and data flow. Taking Dify as a representative example, it provides essential built-in nodes (e.g., Code, If-Else) alongside hundreds of community-driven extensions (e.g., Google Search), enabling

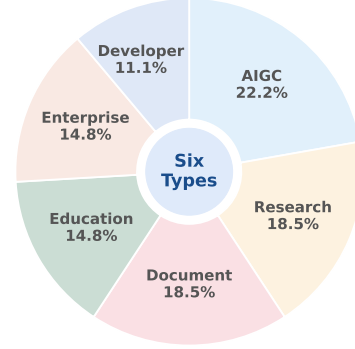


Figure 2: Distribution of task types in Chat2Workflow. The benchmark covers six domains: AIGC, Research, Document, Education, Enterprise, and Developer.

the resolution of highly diverse and complex tasks.

2.2 Task Formulation

We formalize the generation task of the workflow as a multi-turn interaction process. Given a task instruction for the current round and historical interaction dialogues, we aim to enable language agents to generate graph-structured workflows represented in JSON, which describes the nodes and connectivity in a structured manner. Formally, given the instruction q , historical dialogues H and language agents $\mathcal{M}_\theta$, the workflow generation can be represented as:

$$\mathcal{G}(\mathcal{V}, \mathcal{E}) \leftarrow \mathcal{M}_\theta(q, H), \quad (1)$$

where $\mathcal{G}$ is a Directed Acyclic Graph with node list $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ comprising nodes or tools with predefined parameters populated and edges $\mathcal{E} = \{(v_i, p, v_j)\}, 1 \leq i \neq j \leq n, p \in \mathbb{Z}_{\geq 0}$ indicating that the source node v_i is connected to the target node v_j through the output branch port p .

Directly generating executable workflows is challenging for LLMs. To address this, we use a Chain-of-Thought (CoT) approach that outputs a specially designed simplified JSON. The model generates selected nodes $\hat{\mathcal{V}}$, planning rationale p , and the JSON representation r , wrapped in `<node_selection>`, `<design_principle>` and `<workflow>` tags. This JSON is then converted into an executable YAML file. It can be formalized as:

$$\mathcal{G}(\mathcal{V}, \mathcal{E}) \leftarrow r \leftarrow \{\hat{\mathcal{V}}, p, r\} \leftarrow \mathcal{M}_\theta(q, H), \quad (2)$$

where $\mathcal{V}$ and $\mathcal{E}$ correspond to the `nodes_info` and `edges` fields in JSON respectively.

2.3 Node Selection and Processing

We selected the 20 most frequently used nodes to capture major real-world scenarios. To reduce

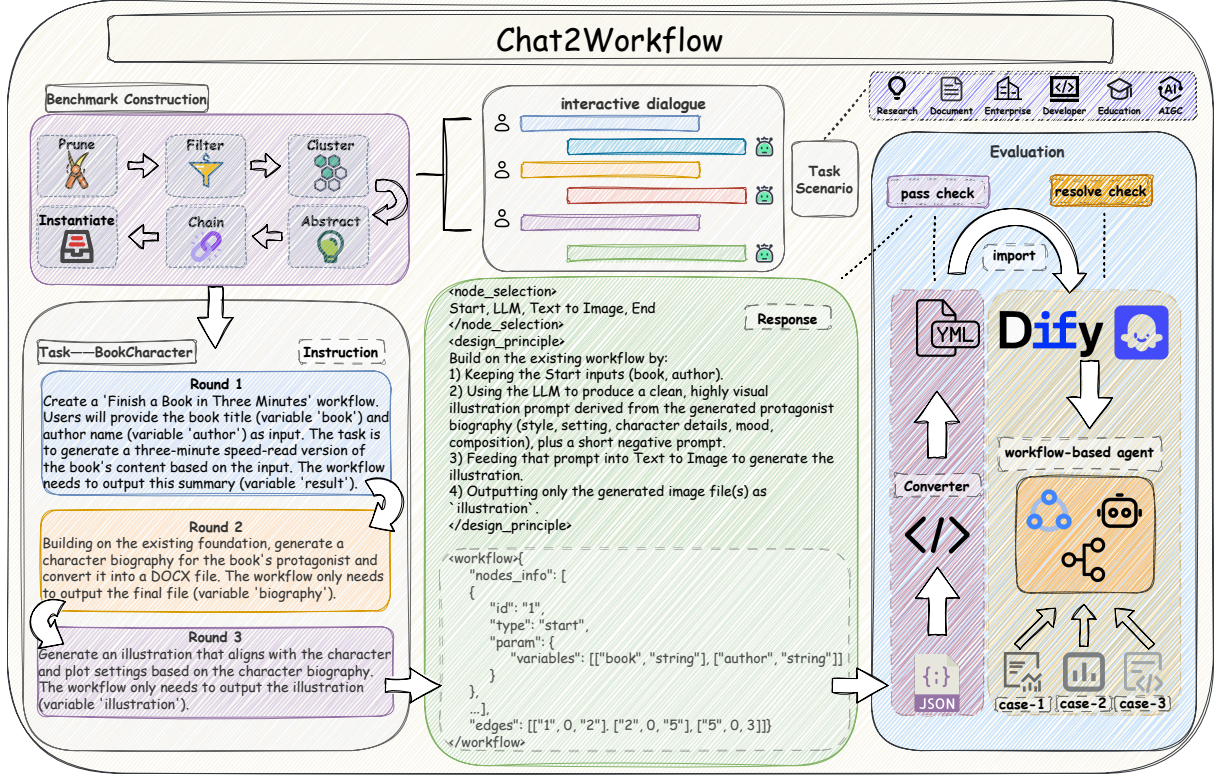


Figure 3: Overview of Chat2Workflow benchmark construction and evaluation framework. **Left:** The process of constructing multi-turn instructions involves six sequential stages: Prune, Filter, Cluster, Abstract, Chain, and Instantiate. **Center:** Users interact with LLMs through dialogue, and the model generates workflows in JSON format with CoT reasoning (node selection, design principles, and structured workflow). **Right:** The workflows span six distinct task scenarios. The JSON workflow is converted to YAML format, uploaded to the platforms for execution, and evaluated against test cases to compute Pass Rate and Resolve Rate.

generation complexity, we simplified the nodes’ input/output (I/O) variable interfaces: the language agent only needs to provide key-value pairs for essential primary variables, while secondary variables default. In order to endow the language agent with the usage of each node and corresponding I/O interfaces, we craft node knowledge base for 20 node types, serving as a crucial part of the system prompt used for understanding the node ecosystem. Based on these principles, we extract and rewrite essential information from official node documents into a predefined structure comprising `<type>`, `<description>`, `<parameter>`, `<referable_variables>`, and `<supplementary_information>`.

2.4 Dataset Construction

Figure 3 outlines our construction and evaluation framework. To build a multi-turn workflow dataset, **three independent CS undergraduate volunteers** processed production-level Dify and Coze configurations collected from official sources and GitHub through a systematic pipeline:

Pruning and Filtering. We first substituted unsupported workflow nodes with functional equivalents or pruned unviable functional components. Workflows with poor testability were discarded.

Clustering and Abstracting. The remaining workflows were clustered into six task scenarios: Research, Document, Enterprise, Developer, Education, and AIGC (details in Appendix A). Within each cluster, workflows were abstracted into natural language representations of their core functions, which are closely tied to essential key nodes.

Chaining. To construct multi-turn tasks (2–4 rounds), volunteers manually chained related workflows, treating each as a single turn. Each round’s instruction sequentially adds, modifies, or deletes functions from the previous round while maintaining logical consistency. We maximized variation across graph topologies, I/O types, and file formats, strictly distinguishing between function modifications and complete overwrites. I/O variables are explicitly formatted in parentheses, and unavoidable key nodes are recorded in advance as hints.

Instantiating. To enable end-to-end evaluation, we instantiated each instruction with three test cases that comprehensively cover historical requirements and multi-branch logic. Test cases were sourced hierarchically: (1) relevant Hugging Face datasets adapted to our I/O interfaces, (2) real-world internet materials, and (3) LLM-synthesized data. Reference answers are provided for deterministic outputs, whereas open-ended general cases consist solely of the provided inputs.

2.5 Quality Control and Dataset Statistics

We implement a strict **cross-validation review process**, with **three volunteers** auditing each other’s data to ensure correctness, completeness and privacy. To pass, the instructions must allow the target workflow’s key nodes and core functions to be manually recreated without ambiguity, exactly matching the creator’s intent. Cases with invalid intermediate steps, time-varying sources, or incoherent behaviors are systematically removed.

Following this rigorous validation, we curated Chat2Workflow, a high-quality dataset consisting of 27 tasks with 79 multi-turn instructions. Accompanied by 237 test instances (three per instruction), it robustly assesses whether generated workflows adhere to complex execution requirements.

2.6 Evaluation Protocol

Unlike standard text or image generation, workflows must be evaluated not only on their representational format but also on their executability and outcomes. Thus, we adopt a two-stage progressive evaluation: *pass rate* assesses structural and logical validity, while *resolve rate* evaluates functional execution correctness.

Pass rate measures the proportion of subtasks that successfully pass the sequential checks in Algorithm 1. Specifically, a subtask passes if its JSON representation aligns with the CoT sequence and predefined variables, and converts into an importable YAML file. It is defined as:

$$\%Pas. = \frac{N_{\text{success_subtask}}}{N_{\text{total_subtask}}}, \quad (3)$$

where $N_{\text{success_subtask}}$ and $N_{\text{total_subtask}}$ denote the number of passing subtasks and the total number of subtasks, respectively.

Resolve rate measures the proportion of test cases where the generated workflow correctly executes and produces outputs satisfying all instruction

Algorithm 1: Pass Rate Pipeline

Data: Model response $\mathcal{R}$, Reference variables $\mathcal{V}_{ref}$,
Key nodes N_{key}
Notation: Selected nodes $\hat{\mathcal{V}}$, Planning rationale p ,
JSON file r , YAML file F_{yaml} , Workflow $\mathcal{G}$
// Step 1: Format Check
 $\hat{\mathcal{V}}, p, r \leftarrow \text{PARSE}(\mathcal{R})$;
if $\mathcal{R}$ lacks required tags **or** JSON r not parsable **then**
 return *False*;
// Step 2: Conversion & Import
 $F_{yaml} \leftarrow \text{JSONTOYAML}(r)$;
if conversion fails **then**
 return *False*;
 $\mathcal{G} \leftarrow \text{IMPORT}(F_{yaml})$;
// Step 3: Variable Consistency
if $\text{EXTRACTVARS}(r) \neq \mathcal{V}_{ref}$ **then**
 return *False*;
// Step 4: Logical Validity
if $f_{LLM}(\hat{\mathcal{V}}, p, r)$ indicates inconsistency **then**
 return *False*;
if $f_{LLM}(N_{key}, \hat{\mathcal{V}})$ indicates $N_{key} \not\subseteq \hat{\mathcal{V}}$ **then**
 return *False*;
return *True*;

requirements in Algorithm 2. It is defined as:

$$\%Res. = \frac{N_{\text{success_case}}}{N_{\text{total_case}}}, \quad (4)$$

where $N_{\text{success_case}}$ and $N_{\text{total_case}}$ represent the number of successful test cases and the total number of test cases, respectively.

Algorithm 2: Resolve Rate Pipeline

Data: Workflow $\mathcal{G}$, instructions $\mathcal{I}$, Test case
 $\mathcal{T} = \{\text{input}, \text{ref_output}\}$
Notation: Execution output $\mathcal{O}$
// Step 1: Execution Check
 $\mathcal{O} \leftarrow \text{EXECUTE}(\mathcal{G}, \mathcal{T}.\text{input})$;
if execution fails with errors **or** $\mathcal{O} = \emptyset$ **then**
 return *False*;
// Step 2: Output Validation
 $\mathcal{O}_{file}, \mathcal{O}_{text} \leftarrow \text{SPLITOUTPUT}(\mathcal{O})$;
if $\mathcal{O}_{file} \neq \emptyset$ **and** file types/extensions mismatch **then**
 return *False*;
if $\mathcal{O}_{text} = \emptyset$ **then**
 return *True*;
if $f_{LLM}(\mathcal{I}, \mathcal{O}_{text}, \mathcal{T}.\text{input}, \mathcal{T}.\text{ref_output})$ confirms
 all requirements met **then**
 return *True*;
return *False*;

We employ GPT-5.1 for evaluation, sampling 355 instances for the pass stage and 463 for the resolve stage. To ensure reliability, **three independent volunteers** reviewed the LLM judge’s reasoning and verdicts. Discrepancies were handled by requiring strict unanimity: the agreement

Model	Research		Document		Enterprise		Developer		Education		AIGC		Average	
	%Pas.	%Res.	%Pas.	%Res.	%Pas.	%Res.	%Pas.	%Res.	%Pas.	%Res.	%Pas.	%Res.	%Pas.	%Res.
<i>Closed-Sourced</i>														
GPT-5.1	48.89	41.48	53.33	40.74	50.00	33.33	20.83	8.33	36.36	19.19	57.41	45.06	47.26	34.46
GPT-5.2	68.89	53.33	66.67	50.37	86.11	58.33	70.83	38.89	75.76	54.55	48.15	32.10	67.51	47.40
Claude-Sonnet-4.5	68.89	56.30	75.56	62.22	61.11	27.78	75.00	31.94	90.91	60.61	62.97	41.97	71.31	47.96
Gemini-3-Pro-Preview	77.78	62.96	82.22	69.63	86.11	64.82	70.83	44.44	90.91	59.60	74.08	54.32	80.17	60.20
<i>Open-Sourced</i>														
Qwen-3-8B	40.00	21.48	8.89	4.45	8.33	0.00	12.5	1.39	3.03	0.00	7.41	6.79	13.92	6.61
Qwen-3-14B	26.67	18.52	33.33	14.08	13.89	0.00	20.83	11.11	27.27	0.00	35.18	22.22	27.43	12.38
Qwen-3-32B	31.11	22.22	28.89	13.33	30.56	5.55	50.00	15.28	33.33	23.23	48.15	29.63	36.71	19.13
Qwen-3-235B-A22B	35.55	32.59	51.11	34.07	19.45	1.85	62.50	20.83	39.39	29.29	42.59	23.46	40.93	24.47
Qwen-3-Coder-480B-A35B-Instruct	53.33	29.63	57.78	28.89	25.00	1.85	25.00	0.00	45.45	17.17	37.03	23.46	42.19	19.13
GLM-4.6	62.22	57.04	60.00	39.26	63.89	24.07	45.83	5.55	72.73	25.25	64.82	39.51	62.45	35.02
GLM-4.7	80.00	54.82	62.22	45.18	72.22	44.45	41.67	19.45	78.79	52.53	70.37	50.61	69.20	46.55
Deepseek-V3.1	51.11	42.22	51.11	34.07	61.11	24.07	20.83	2.78	45.45	32.32	62.96	40.12	51.48	32.07
Deepseek-V3.2	51.11	40.74	46.67	36.29	27.78	1.85	16.67	4.17	42.42	22.22	57.41	39.51	43.46	27.43
Kimi-K2-Instruct	51.11	40.74	37.78	27.40	11.11	8.33	29.17	5.56	21.21	13.13	51.85	35.19	36.29	24.61
Kimi-K2-Thinking	55.56	44.44	66.67	46.67	52.78	25.93	62.50	26.39	27.27	15.15	72.22	43.83	57.81	36.00

Table 1: Experimental results across six task domains for 4 closed-source and 11 open-source models. Pass Rate (%Pas.) measures format correctness, while Resolve Rate (%Res.) measures actual problem-solving capability. The best results within each model series are marked in **bold**.

rate is defined as the percentage of cases where all three evaluators independently confirmed that the LLM strictly followed the evaluation prompt’s constraints (format, content, logic, and instruction following). Under this strict protocol, the LLM achieved 97.18% agreement for the pass stage and 93.09% for the resolve stage. Cross-model validation with DeepSeek-V3 demonstrates consistent stability (96.62% and 91.79%). However, it prioritizes formal compliance over deep semantic accuracy, unlike the stringent GPT-5.1. The exact prompts are detailed in Appendix (Figures 8 and 9).

3 Experiments

3.1 Experimental Setup

To thoroughly assess workflow generation capabilities, we validate 15 representative models on the Chat2Workflow benchmark, reporting the average results across **three independent runs**:

- 1) Four **closed-source** LMs: GPT-{5.1, 5.2}, Claude-Sonnet-4.5, and Gemini-3-Pro-Preview.
- 2) Eleven **open-source** LMs: Qwen-3-{8B, 14B, 32B}, Qwen3-235B-A22B, Qwen3-Coder-480B-A35B-Instruct, GLM-{4.6,4.7}, DeepSeek-{V3.1,V3.2}, Kimi-K2-{Instruct, Thinking}.

Furthermore, to explore the performance upper bound, we select two representative models (GPT-5.1 and GPT-5.2) for an advanced agentic evaluation, which is detailed in Section 3.5.

The experiments are conducted on the 1.9.2 version of Dify. More details are in Appendix B.

3.2 Main Results

Table 1 presents our experimental results, which we use to analyze the following research questions.

Q1: What is the gap between the workflow pass rate and resolve rate? For each task, we report both the pass rate and resolve rate to evaluate format compliance and practical usability, respectively. All models exhibit a lower resolve rate than pass rate. GLM-4.6 shows the largest disparity, averaging 27.43% and surging to 47.48% in the Education scenario. Although Kimi-K2-Instruct maintains a narrow gap of 11.68%, its absolute performance remains poor compared to the highest resolve rate (60.20%), particularly dropping in the Developer scenario. These results demonstrate that a syntactically legal workflow does not guarantee successful execution. Pass rates are necessary but insufficient, often inflating performance evaluations.

Q2: To what extent do existing LLM agents approach the capabilities of real workflow design experts? Our benchmark largely represents real-world scenarios through its use of typical workflows, interactive creation mechanisms, and file I/O requirements. Evaluated purely on end-to-end task resolution, Gemini-3-Pro-Preview performs best with an absolute resolve rate of 60.20%. However, It remains far below the threshold for a reliable, deployable planner. Even the best open-source model, GLM-4.7, achieves only a 46.55% resolve rate and shows significant weakness in Developer tasks. Given our limited set of node

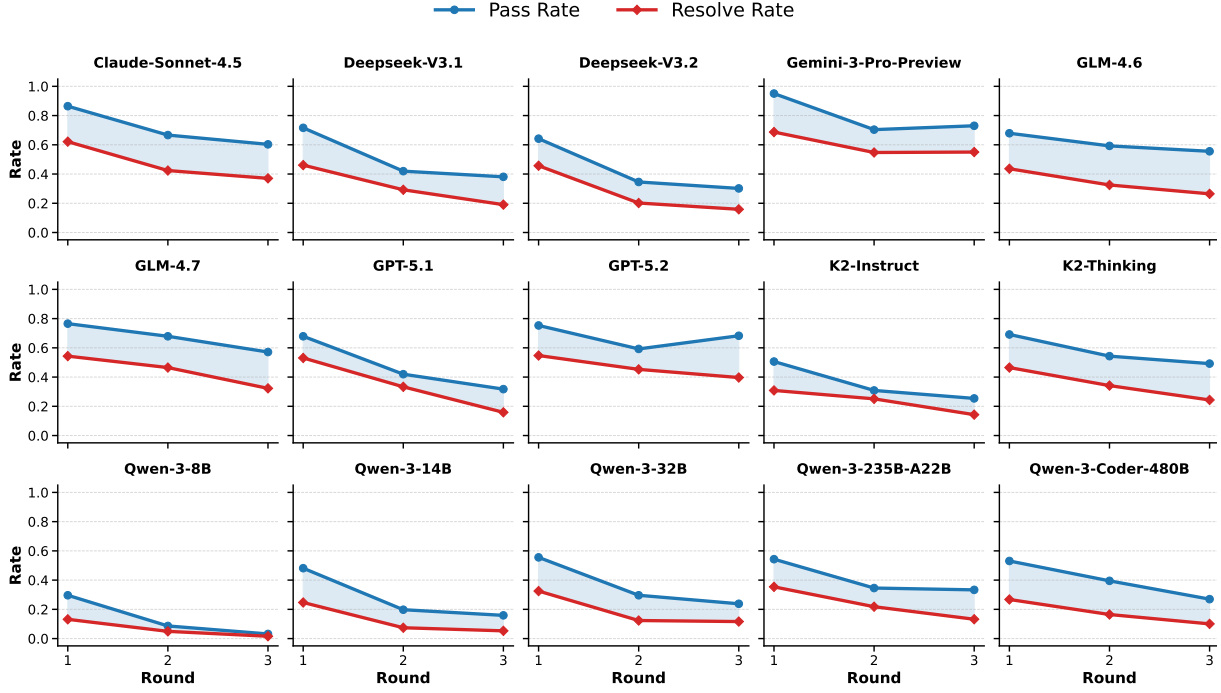


Figure 4: Performance degradation across dialogue rounds. We show the Pass Rate and Resolve Rate for all 15 models across the first three dialogue rounds. Most models exhibit a steady decline in both metrics as the number of interaction rounds increases, indicating the challenge of maintaining workflow quality under evolving requirements.

types, real-world workflow creation is inherently more complex. Consequently, we anticipate that even leading models will struggle further when confronted with larger node vocabularies.

Q3: How do scale, alignment, and reasoning impact workflow generation performance?

1) **Scale.** Increasing model parameters directly enhances performance. The Qwen-3 series demonstrates stable and significant improvements as its size scales from 8B to 32B and ultimately to 235B. 2) **Alignment.** While post-training improves format compliance, it does not guarantee problem-solving efficacy. For instance, in the Developer scenario, GLM-4.6 achieves a higher pass rate than GLM-4.7 (45.83% vs. 41.67%) but a drastically lower resolve rate (5.55% vs. 19.45%). Thus, optimizing solely for formatting may not effectively boost actual workflow usability. 3) **Reasoning.** Models equipped with reasoning capabilities consistently outperform standard instruction-tuned variants. Despite its massive scale, Qwen-3-Coder-480B-A35B-Instruct underperforms the smaller Qwen-3-235B-A22B in resolve rate (19.13% vs. 24.47%). Similarly, Kimi-K2-Thinking surpasses its instruct counterpart across all task domains, proving that explicit reasoning mechanisms are crucial for effective and robust workflow generation.

Q4: What shall we do to enhance the workflow generation capability of LLM agents?

Analyzing failures from both the pass stage (see Figure 5 for the error distribution) and the resolve stage, we categorize the underlying errors into four primary dimensions. 1) **Format.** Syntactic and parsing rule violations, such as unclosed JSON brackets, redundant code fences, and improper character escaping. 2) **Graph.** Topological and structural flaws, including forward references violating topological sorting, hallucinated node connections, isolated nodes, and invalid cross-boundary edges connecting the inner and outer canvases of an iteration node. 3) **Semantics.** Cognitive misalignment regarding operational rules and node functionalities, manifested as failures to adhere to specific node configurations, violations of environment constraints, incorrect node types, or incompatible variable references. 4) **Consistency.** Functional or logical contradictions within the generation. This includes misalignments between the workflow’s actual function and the user’s core intent, mismatches with requested I/O variables, and logical discrepancies among different components of the model response or between consecutive turns. Therefore, addressing these critical bottlenecks is key to advancing LLM

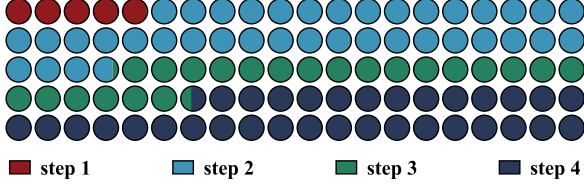


Figure 5: Error probabilities across the four steps of the pass stage (corresponding to Algorithm 1).

agents’ workflow generation capabilities.

3.3 Analysis of User Requirement Changes from Chat Interactions

Each task consists of two to four dialogue rounds. In each round, workflow requirements are iteratively modified based on historical context, presenting a significant challenge to the models’ long-horizon instruction following.

To ensure robust representations, we analyze the evaluation metrics over the first three rounds (Figure 4), omitting the fourth round due to its limited sample size. For most models, workflow quality consistently declines as interactions increase, corroborating the difficulty of handling cumulative modifications. The slight improvement observed for GPT-5.2 in the third round merely results from a changing denominator, as two-round tasks drop out of the calculation scope.

Furthermore, the degradation curve flattens over time. Rather than degrading at a uniform pace, the decline diminishes in later rounds. This indicates that workflows surviving the initial modifications act as a robust baseline. Once a model grasps the core logic in early turns, its capacity to accommodate further incremental changes stabilizes, reflecting a shift from drastic qualitative failures to minor quantitative errors.

3.4 Case Analysis

We take the StudyPlanner task of Education scenario as an example for analysis. The instructions for the first two rounds are in Appendix C.

Figure 6 illustrates the second-round responses from different models. Specifically, the visual workflows generated by GPT-5.2 on the **Diffy** and **Coze** platforms are shown in Figures 10 and 11 of the Appendix. 1) **Kimi-K2-Instruct** generates an invalid edge. Node 5 is an `iteration` node that internally contains a sub-workflow starting with `iteration-start`. Despite documentation explicitly restricting their relationship to inclusion

only, the model erroneously connects them with an edge, rendering the workflow unexecutable. 2) **GPT-5.2** successfully resolves the case by generating a valid, executable workflow. 3) **GLM-4.6** exhibits logical inconsistencies. It utilizes the `code` and `iteration-start` nodes in the final workflow without declaring them in `<node_selection>`, indicating a flawed and incoherent workflow creation logic.

3.5 An Error-Driven Agentic Baseline

To address the previously identified bottlenecks, we propose an error-driven agentic baseline to explore the performance upper bound of LLMs on the benchmark. Implemented via the **OpenCode** framework, this approach transitions from vanilla zero-shot prompting to a structured **SKILL-based paradigm**, explicitly incorporating task-critical guidelines, as well as rules for multi-turn interactions and variable references.

To mitigate context decay in subsequent interaction turns, we dynamically extract **Variable Summaries** from preceding workflow iterations as supplementary context. Crucially, we construct a robust execution loop featuring a **5-attempt retry mechanism** with comprehensive structural and semantic verification. Upon failure, the framework triggers targeted **Auto-Repair Modules** to rectify the four most prevalent errors: redundant code fences, JSON decoding failures, topological sorting violations, and node selection inconsistencies. As shown in Figure 7, the agentic framework yields substantial improvements, boosting Resolve Rate for GPT-5.1 and GPT-5.2 by absolute margins of 4.78% and 6.05%, respectively.

4 Related work

4.1 LLM-based Agents

LLM-based agents have demonstrated exceptional capabilities in solving complex real-world problems with the aid of general tools (Schick et al., 2023; Patil et al., 2024), collaborative frameworks (Hong et al., 2024; Qian et al., 2023), and extensive API repositories (Qin et al., 2024; Tang et al., 2023; Zhang et al., 2024a; Yang et al., 2024). Many prompt-based methods (Wei et al., 2022; Yao et al., 2022; Zhu et al., 2026) are proven effective for improving performance. However, these agent methods or frameworks mainly focus on end-to-end effects, paying little attention to the requirements and norms of the intermediate processes in solving

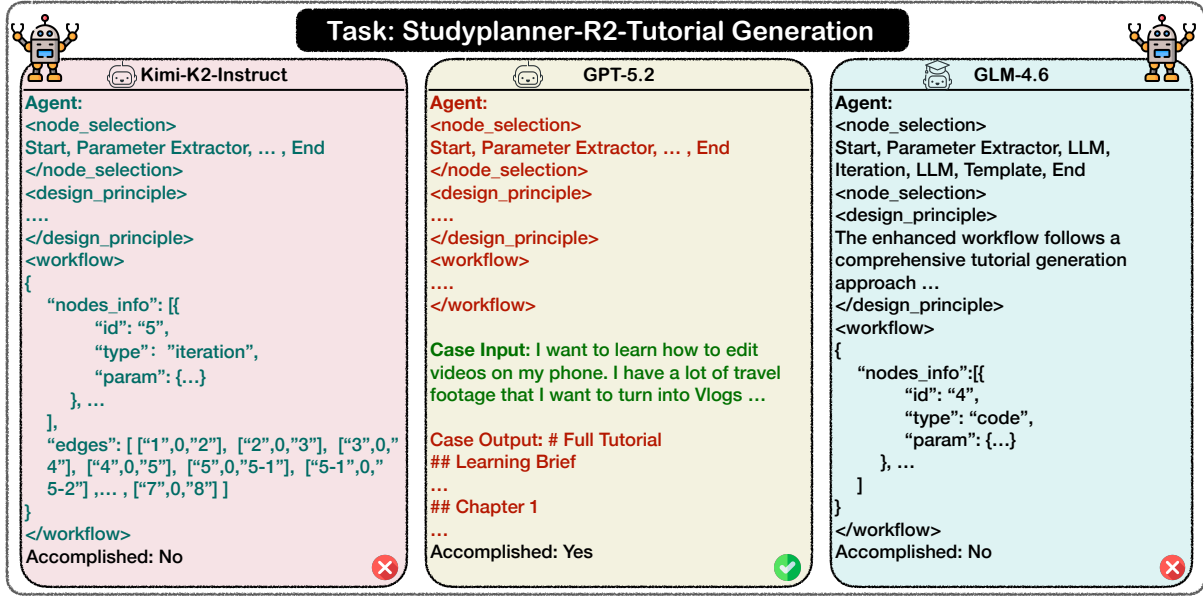


Figure 6: Bad case analysis for the StudyPlanner task. We compare outputs from three representative models: Kimi-K2-Instruct produces an invalid edge connection in the iteration node; GPT-5.2 generates a correct and executable workflow; GLM-4.6 outputs logically inconsistent node declarations. It reveals common failure patterns.

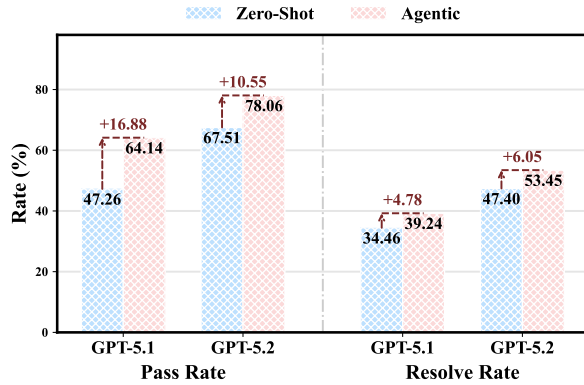


Figure 7: Performance comparison demonstrating the improvements of the proposed Agentic baseline over the Zero-Shot setting for GPT-5.1 and GPT-5.2.

tasks, such as planning and reasoning paths, which is not conducive to stable and reliable execution, as well as the reproduction of results across various domains (Ge et al., 2024; Xie et al., 2024; Liu et al., 2024; Yang et al., 2026; Liang et al., 2026).

4.2 Workflow and Language Agent Planning

As an intermediate state, workflows bridge the gap between high-level goals and executable steps. By decomposing task into several executable atomic steps and organizing them into execution plans based on formal logical relationships (van der Aalst, 1997; Dijkman et al., 2008), the workflow

achieves complete transparency of the reasoning path, effectively enhancing the interpretability and reliability of the results. Additionally, prior structural constraints facilitate complex task logic via established patterns (van der Aalst et al., 2003; Commoner et al., 1971) and robotic process automation (Ivančić et al., 2019; Hofmann et al., 2020). Although early workflows relied on labor-intensive manual design to avoid hallucinations, recent research has shifted towards automated generation using LLMs (Shen et al., 2023a; Zeng et al., 2023). This includes iterative synthesis (Zhang et al., 2024b; Li et al., 2024) and knowledge-augmented planning frameworks (Ye et al., 2023; Zhu et al., 2024; Wornow et al., 2024; Huang et al., 2024). Appendix D provides more related work.

5 Conclusion

We introduce Chat2Workflow, a benchmark for evaluating LLMs’ ability to generate deployable visual workflows from natural language, and show that even when augmented with an agentic framework, current models remain fragile under structural constraints and requirement changes. Chat2Workflow can provide both a realistic testbed and a concrete target for future research on automated workflow engineering, and will help bridge the gap between language model-driven workflow design and industrial deployment.

Limitations

Despite its contributions, Chat2Workflow has several limitations. First, while the high-quality dataset is manually verified, the current scale may not encompass the near-infinite variety of logic found in complex industrial business processes. Second, we simplified node interfaces to prioritize executability, which may not fully capture the intricate parameter configurations required in some real-world deployments. Finally, the current system incorporates only 20 high-frequency node types. These specific nodes were selected as they effectively cover most standard workflow scenarios. However, a wide range of valuable tools remain to be incorporated.

References

- Zehui Chen, Weihua Du, Wenwei Zhang, Kuikun Liu, Jiangning Liu, Miao Zheng, Jingming Zhuo, Songyang Zhang, Dahua Lin, Kai Chen, and 1 others. 2024. T-eval: Evaluating the tool utilization capability of large language models step by step. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9510–9529.
- Frederic G. Commoner, Anatol W. Holt, Shimon Even, and Amir Pnueli. 1971. [Marked directed graphs](#). *J. Comput. Syst. Sci.*, 5(5):511–523.
- Remco M. Dijkman, Marlon Dumas, and Chun Ouyang. 2008. [Semantics and analysis of business process models in BPMN](#). *Inf. Softw. Technol.*, 50(12):1281–1294.
- Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. Workflowllm: Enhancing workflow orchestration capability of large language models. *arXiv preprint arXiv:2411.05451*.
- Yingqiang Ge, Wenyue Hua, Kai Mei, Juntao Tan, Shuyuan Xu, Zelong Li, Yongfeng Zhang, and 1 others. 2024. Openagi: When llm meets domain experts. *Advances in Neural Information Processing Systems*, 36.
- Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. *arXiv preprint arXiv:2403.07714*.
- Peter Hofmann, Caroline Samp, and Nils Urbach. 2020. Robotic process automation. *Electronic markets*, 30(1):99–106.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. [Metagpt: Meta programming for A multi-agent collaborative framework](#). In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Tian Huang, Chun Yu, Weinan Shi, Zijian Peng, David Yang, Weiqi Sun, and Yuanchun Shi. 2024. Promptpa: Generating robotic process automation on smartphones from textual prompts. *arXiv preprint arXiv:2404.02475*.
- Lucija Ivančić, Dalia Suša Vugec, and Vesna Bosilj Vukšić. 2019. Robotic process automation: systematic literature review. In *Business Process Management: Blockchain and Central and Eastern Europe Forum: BPM 2019 Blockchain and CEE Forum, Vienna, Austria, September 1–6, 2019, Proceedings 17*, pages 280–295. Springer.
- Yash Kumar Lal, Vanya Cohen, Nathanael Chambers, Niranjana Balasubramanian, and Raymond J. Mooney. 2024. [Cat-bench: Benchmarking language model understanding of causal and temporal dependencies in plans](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 16-16, 2024*, pages 19336–19354. Association for Computational Linguistics.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms. *arXiv preprint arXiv:2304.08244*.
- Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. 2024. [Autoflow: Automated workflow generation for large language model agents](#). *CoRR*, abs/2407.12821.
- Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, and 30 others. 2026. [Skillnet: Create, evaluate, and connect AI skills](#). *CoRR*, abs/2603.04448.
- Xiao Liu, Tianjie Zhang, Yu Gu, Iat Long Iong, Yifan Xu, Xixuan Song, Shudan Zhang, Hanyu Lai, Xinyi Liu, Hanlin Zhao, Jiadai Sun, Xinyue Yang, Yu Yang, Zehan Qi, Shuntian Yao, Xueqiao Sun, Siyi Cheng, Qinkai Zheng, Hao Yu, and 11 others. 2024. [Visualagentbench: Towards large multimodal models as visual foundation agents](#). *CoRR*, abs/2408.06327.
- Melissa Z Pan, Negar Arabzadeh, Riccardo Cogo, Yuxuan Zhu, Alexander Xiong, Lakshya A Agrawal, Huanzhi Mao, Emma Shen, Sid Pallerla, Liana Patel, and 1 others. 2025. Measuring agents in production. *arXiv preprint arXiv:2512.04123*.

- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565.
- Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. [Communicative agents for software development](#). *CoRR*, abs/2307.07924.
- Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2024. Benchmarking agentic workflow generation. *arXiv preprint arXiv:2410.07869*.
- Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2025. Benchmarking agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. [Toolllm: Facilitating large language models to master 16000+ real-world apis](#). In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.
- Haiyang Shen, Yue Li, Desong Meng, Dongqi Cai, Sheng Qi, Li Zhang, Mengwei Xu, and Yun Ma. 2024. Shortcutsbench: A large-scale real-world benchmark for api-based agents. *arXiv preprint arXiv:2407.00132*.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023a. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180.
- Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. 2023b. [Taskbench: Benchmarking large language models for task automation](#). *CoRR*, abs/2311.18760.
- Yuchen Shi, Siqi Cai, Zihan Xu, Yuei Qin, Gang Li, Hang Shao, Jiawei Chen, Deqing Yang, Ke Li, and Xing Sun. 2025. Flowagent: Achieving compliance and flexibility for workflow agents. *arXiv preprint arXiv:2502.14345*.
- Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, and Le Sun. 2023. [Toolalpaca: Generalized tool learning for language models with 3000 simulated cases](#). *CoRR*, abs/2306.05301.
- Wil M. P. van der Aalst. 1997. [Verification of workflow nets](#). In *Application and Theory of Petri Nets 1997, 18th International Conference, ICATPN '97, Toulouse, France, June 23-27, 1997, Proceedings*, volume 1248 of *Lecture Notes in Computer Science*, pages 407–426. Springer.
- Wil M. P. van der Aalst, Arthur H. M. ter Hofstede, Bartek Kiepuszewski, and Alistair P. Barros. 2003. [Workflow patterns](#). *Distributed Parallel Databases*, 14(1):5–51.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, and 1 others. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Michael Wornow, Avanika Narayan, Krista Opsahl-Ong, Quinn McIntyre, Nigam H Shah, and Christopher Re. 2024. Automating the enterprise with foundation models. *arXiv preprint arXiv:2405.03710*.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, and 1 others. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101.
- Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024. Travelplanner: A benchmark for real-world planning with language agents. *arXiv preprint arXiv:2402.01622*.
- Zhenran Xu, Yiyu Wang, Xue Yang, Longyue Wang, Weihua Luo, Kaifu Zhang, Baotian Hu, and Min Zhang. 2025. [Comfyui-r1: Exploring reasoning models for workflow generation](#). *Preprint*, arXiv:2506.09790.
- Ke Yang, Jiateng Liu, John Wu, Chaoqi Yang, Yi R. Fung, Sha Li, Zixuan Huang, Xu Cao, Xingyao Wang, Yiquan Wang, Heng Ji, and Chengxiang Zhai. 2024. [If LLM is the wizard, then code is the wand: A survey on how code empowers large language models to serve as intelligent agents](#). *CoRR*, abs/2401.00812.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. 2026. [Autoskill: Experience-driven lifelong learning via skill self-evolution](#). *CoRR*, abs/2603.01145.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

Junjie Ye, Guanyu Li, Songyang Gao, Caishuang Huang, Yilong Wu, Sixian Li, Xiaoran Fan, Shihan Dou, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. [Tookeyes: Fine-grained evaluation for tool learning capabilities of large language models in real-world scenarios](#). *CoRR*, abs/2401.00741.

Yining Ye, Xin Cong, Shizuo Tian, Jiannan Cao, Hao Wang, Yujia Qin, Yaxi Lu, Heyang Yu, Huadong Wang, Yankai Lin, and 1 others. 2023. Proagent: From robotic process automation to agentic process automation. *arXiv preprint arXiv:2311.10751*.

Zhen Zeng, William Watson, Nicole Cho, Saba Rahimi, Shayleen Reynolds, Tucker Balch, and Manuela Veloso. 2023. Flowmind: automatic workflow generation with llms. In *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 73–81.

Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Awalganekar, Rithesh Murthy, Eric Hu, Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, and 3 others. 2024a. [xlam: A family of large action models to empower ai agent systems](#). *Preprint*, arXiv:2409.03215.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2024b. [Aflow: Automating agentic workflow generation](#). *Preprint*, arXiv:2410.10762.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, and 1 others. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2).

Huaxiu Steven Zheng, Swaroop Mishra, Hugh Zhang, Xinyun Chen, Minmin Chen, Azade Nova, Le Hou, Heng-Tze Cheng, Quoc V. Le, Ed H. Chi, and Denny Zhou. 2024. [NATURAL PLAN: benchmarking llms on natural language planning](#). *CoRR*, abs/2406.04520.

Yuqi Zhu, Shuofei Qiao, Yixin Ou, Shumin Deng, Ningyu Zhang, Shiwei Lyu, Yue Shen, Lei Liang, Jinjie Gu, and Huajun Chen. 2024. [Knowagent: Knowledge-augmented planning for llm-based agents](#). *CoRR*, abs/2403.03101.

Yuqi Zhu, Jintian Zhang, Zhenjie Wan, Yujie Luo, Shuofei Qiao, Zhengke Gui, Da Zheng, Lei Liang, Huajun Chen, and Ningyu Zhang. 2026. Light-thinker++: From reasoning compression to memory management. *arXiv preprint arXiv:2604.03679*.

A Detailed Dataset Description

The description of the task scenarios is as follows:

Research includes 5 tasks, which focus on systematically obtaining, understanding, and integrating knowledge from unstructured data. They typically take papers, books, or research topics as input, emphasizing in-depth reading of the content, logical organization, and critical thinking skills.

Document includes 5 tasks, whose core value lies in transforming raw inputs such as files, images, and tables into structured results that are computable, reusable, and processable downstream. They serve as an important representative of the “AI preprocessing layer” in data processing.

Enterprise includes 4 tasks, which are targeted at real Enterprise scenarios, with the goal of automating, standardizing, and providing decision support for complex business processes through AI. Their inputs are mostly internal or business-related documents from enterprises.

Developer includes 3 tasks, which serve scenarios like software development, system design, and technical understanding. The core objective is to lower the threshold for technical understanding and development and enhance engineering efficiency.

Education includes 4 tasks, which revolve around the complete life cycle of “teaching and learning”, with the goal of building a closed-loop system from learning planning, teaching content generation, assessment to feedback.

AIGC includes 6 tasks, which are positioned at the core of “generating directly usable content or materials from 0 to 1”, emphasizing creative expression and multimodal output capabilities. The input is often highly abstract or brief instructions, while the output is complete, publishable, and consumable content results.

B Experimental Settings

Dify platform requires the necessary extensions to be prepared in advance. A set of common models is installed to make the workflows runnable and the results reproducible. Workflows in Dify are natively represented in YAML format. We have developed a code framework for rule-based transformation to convert the JSON format workflow generated by the language agent into valid YAML format. This ensures that workflows can be effectively generated by agents through simplified JSON representation while remaining fully compatible with the Dify execution environment. In addition,

the CoT reasoning sequence is uniformly used as the output method. The node types included in the knowledge base are as follows: Start, End, LLM, Question Classifier, Code, Document Extractor, HTTP Request, If-Else, List Operator, Parameter Extractor, Template, Variable Aggregator, Iteration, Iteration-Start, Text to Speech, Text to Image, Mermaid Converter, Markdown Exporter, Google Search and Echarts. Specifically, the Question Classifier and Parameter Extractor node types are strictly designated to the Qwen-3-Max (qwen3-max), whereas the LLM node type is fixed to the Qwen-3-VL-Plus (qwen3-vl-plus). Text to Image node type are fixed to use the Z-image-Turbo (z-image-turbo) and Text to Speech node type are fixed to use the GPT-4o-Mini-TTS (gpt-4o-mini-tts). For Qwen-3-VL-Plus, temperature is 0.7, max_tokens is 32768, other hyperparameters used during decoding are all set to default values. The system prompt and SKILL.md for workflow generation will be integrated into the GitHub repository soon. As for the Agentic Baseline method, the opencode framework we use is version 1.3.17.

C Instructions for the Studyplanner Task

The instructions for the first two rounds of the Studyplanner task are as follows:

Round 1: Create a learning path planning workflow. The user will provide a descriptive instruction (variable **instruction**) as input. The task is to extract four core fields from the input: interested learning field, learning goals, learning preferences, and average learning duration. Subsequently, based on the extracted information, provide the user with a detailed self-study improvement plan, requiring Markdown format output. The workflow needs to output the self-study plan (variable **plan**).

Round 2: Modify based on the existing foundation. The workflow needs to be able to automatically generate a full set of tutorials. It requires first generating a course syllabus, and then iteratively generating knowledge points by chapter. The content is required to be rigorous, containing rich examples, pros and cons, and precautions. The workflow only needs to output the final tutorial integrated through template conversion (variable **tutorial**).

D More Related Work on Automated Workflow Evaluation

It is crucial to evaluate the quality of the workflow created by LLM. Previous studies have tried to automate workflow evaluation in tool learning scenarios ([Li et al., 2023](#); [Shen et al., 2023b](#); [Xu et al., 2025](#)) and fine-grained planning tasks ([Chen et al., 2024](#); [Ye et al., 2024](#); [Zheng et al., 2024](#)), using metrics to assess format validity and tool consistency. But these works still have some limitations: Firstly, Most of them remain at the stage of abstract expression of the workflow ([Qiao et al., 2024](#)), only checking key elements without considering actual execution effects ([Guo et al., 2024](#)), structural dependencies ([Lal et al., 2024](#)), or platform-specific constraints ([Shen et al., 2024](#)). They may identify obvious format errors but cannot ensure usability. Secondly, these works focus on single-round evaluation, neglecting scenarios where requirements change over time.

You are a strict Dify Workflow evaluator. Your task is to determine whether a workflow design is valid based only on the information provided. Do not make assumptions, infer missing nodes, or introduce any external knowledge.

Inputs

I will provide four sections:

1. `node_selection`: A list or description of nodes selected for the workflow. It represents the nodes the designer claims will be used.
2. `design_principle`: Design principles or constraints for the Dify workflow. It is used to judge overall logical consistency.
3. `workflow`: The JSON string representation of the Dify workflow. You must parse this JSON to extract the nodes that are actually used.
4. `gt_nodes`: A list of mandatory (ground-truth) nodes. These nodes must be included to be considered valid.

Evaluation Rules

All rules must be satisfied for the result to be true. If any rule fails, the result must be false.

Rule 1: Ground-Truth Node Coverage

Check whether node types in `gt_nodes` is a subset of `node_selection`. The node type only needs to appear, and there is no need to consider the frequency of occurrence. If any node type in `gt_nodes` is missing from `node_selection`, immediately return false. Do not continue further checks.

Rule 2: Consistency and Exact Node Matching

You must verify all of the following conditions:

- a. Logical Consistency: `node_selection`, `design_principle`, and `workflow` must be logically consistent. The workflow structure must not violate the stated `design_principle`.
- b. Exact Node Set Matching (Bidirectional Constraint): Extract the actual node set from the workflow JSON. This node set must exactly match the nodes declared in `node_selection`. The node type only needs to appear, and there is no need to consider the frequency of occurrence. Specifically: Nodes not declared in `node_selection` must not appear in workflow. Nodes declared in `node_selection` must appear in workflow.

Note: 1. The type of the "Template" node is "template-transform". 2. Node matching is case-insensitive and ignores hyphens. 3. Treat aliases as identical (e.g., "tts" = "Text to Speech", "text2image" = "Text to Image").

Final Decision Logic

Return true only if Rule 1 and Rule 2 are fully satisfied. If any single condition fails, return false. There is no partial credit or "mostly correct" outcome.

Output Format

Your response must follow this exact format and nothing else:

<reason>

[Explain the evaluation reasoning, stating which rules were satisfied or violated]

</reason>

<result>

[true or false]

</result>

Figure 8: Prompt for evaluating pass rate.

You are an expert evaluator of workflow execution quality. Your task is to judge whether the execution result of a workflow satisfies the requirements of the current round instruction, based on all available information.

Inputs You Will Be Given

You will be provided with the following four parts:

1. queries: A list of historical instructions for creating or modifying the workflow.
 - Each round's instruction is based on the previous ones and may add, modify, or refine workflow behavior.
 - The latest instruction represents the requirements for the current evaluation round.
 - Ignore the input and output requirements for the file in the instruction of query. The file part has been evaluated separately.
2. input: The non-file input variables used in the workflow execution of the current round.
 - This field may be empty.
3. output: The non-file output variables produced by the workflow execution of the current round.
4. reference_answer: A reference answer for the expected result.
 - It may contain minor omissions.
 - Its format may not strictly match the required output format.
 - However, its content is considered correct.
 - This field may be empty.

Evaluation Instructions

Please evaluate whether the workflow execution meets the requirements of the current round instruction, following these principles:

1. Instruction Alignment
 - Identify the requirements implied or explicitly stated in the latest instruction within queries.
 - Consider earlier instructions only insofar as they remain valid and are not overridden.
2. Information Utilization
 - Use all available information (queries, input, output, reference_answer) for judgment.
 - Both of input and reference_answer may be empty; this alone does not imply failure.
3. Output Correctness Criteria
 - If output exists, judge whether its content and format satisfy the current instruction, except for the requirements related to the files.
 - If reference_answer exists, use it as a semantic reference rather than a strict template.
4. Handling Missing or Empty Fields
 - If outputs clearly contradict the instruction, the result should be false, except for the requirements related to the files.
 - If reference_answer is empty, rely on instruction compliance and logical consistency only.
5. Final Judgment Rule
 - If the workflow execution reasonably fulfills the intent and requirements of the current round instruction (except for the requirements related to the files), return true.
 - Otherwise, return false.

****Note:** This rule takes precedence over all the above — The file variable part in the instruction requirements, whether as input or output, including images, documents, and audio will not be provided in this evaluation. So it cannot be used as the basis for a judgment of 'False'!!! For example, if the instruction or query requires the output of a PDF document but the output does not include the document, this point should be ignored at this time to check whether other aspects meet the requirements. ******

Output Format

Your response must follow this exact format and nothing else:

<reason>

[Explain the evaluation reasoning, stating which rules were satisfied or violated]

</reason>

<result>

[true or false]

</result>

Figure 9: Prompt for evaluating resolve rate.

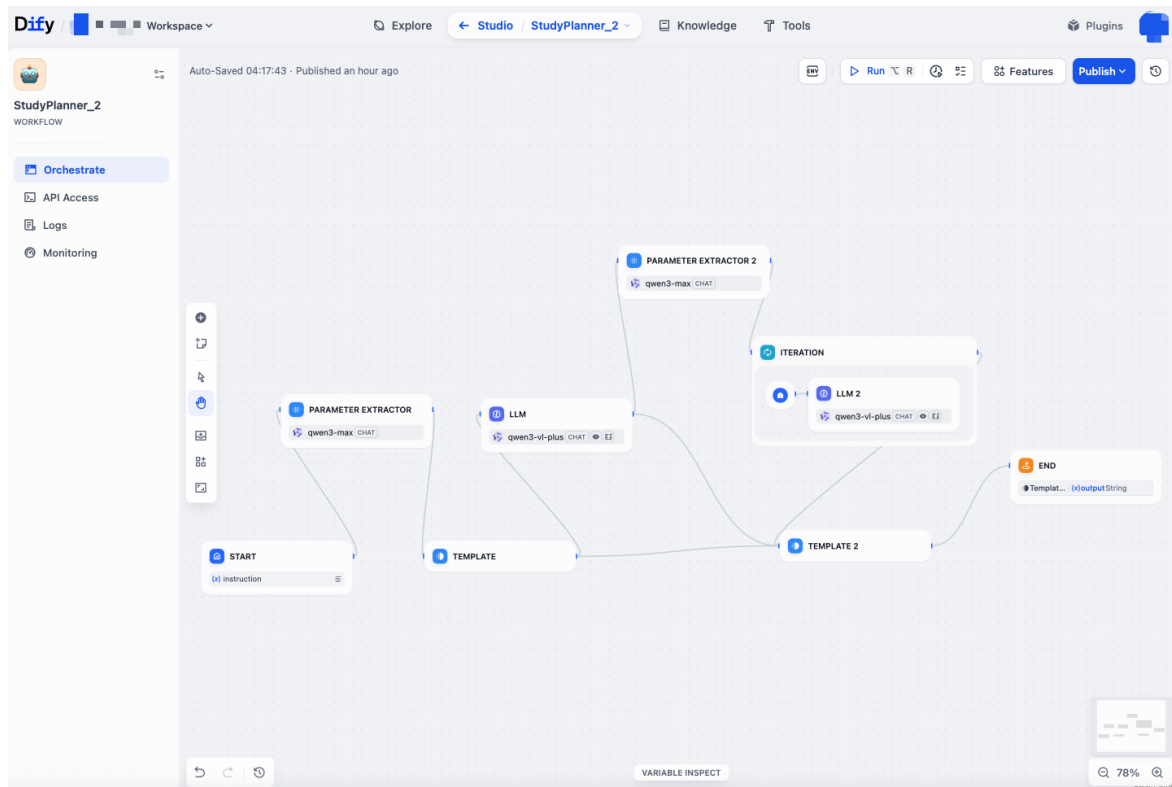


Figure 10: The Dify workflow generated by GPT-5.2 in the second round of the Studyplanner task.

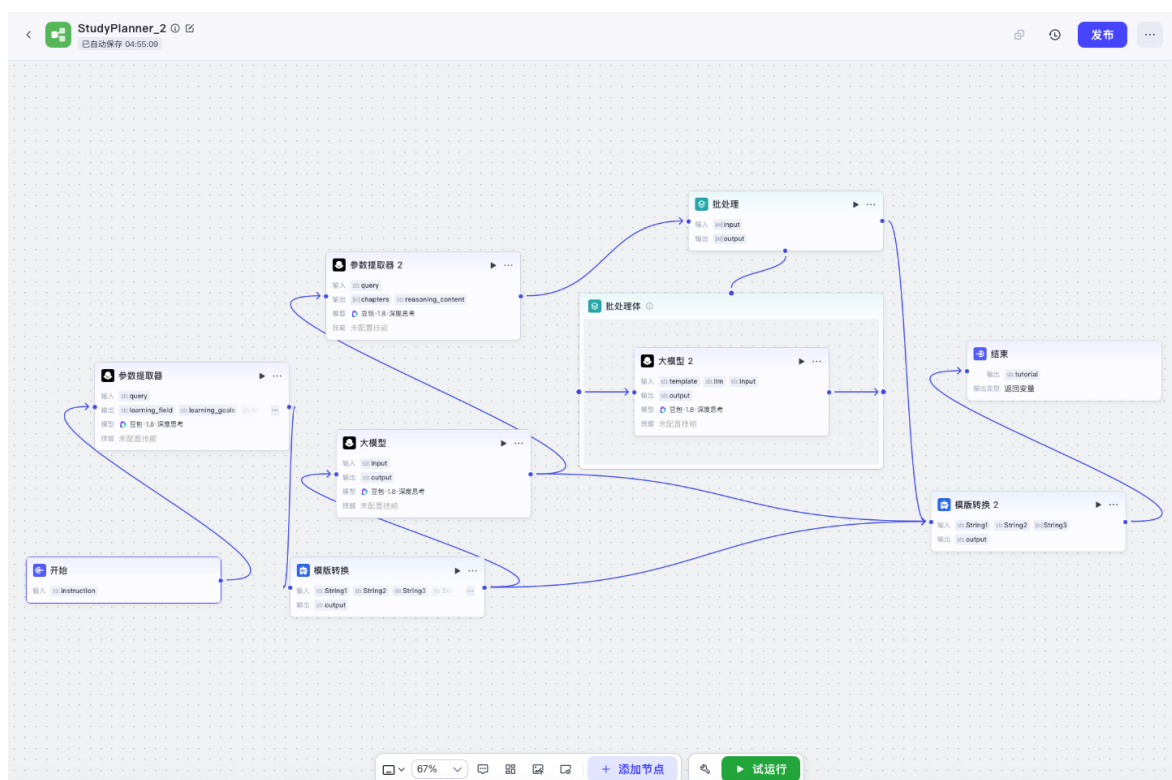


Figure 11: The Coze workflow generated by GPT-5.2 in the second round of the Studyplanner task.